Assignment 04 Report

Course

CSE323: Advanced Embedded Systems Design
CSE411/CSE411s: Real-Time and Embedded Systems Design

Assignment

Assignment 04: Forced Context Switching with `portYIELD_FROM_ISR()`

Objective

The objective of this assignment is to demonstrate the effect of forced context switching from inside an interrupt service routine in FreeRTOS. The required comparison is between a version that calls `portYIELD_FROM_ISR()` and a version that does not call it. The expected difference is that when the macro is used, the awakened high-priority task should run immediately after the ISR. Without the macro, the task should be delayed until the next scheduling opportunity.

System Design

The implemented system contains a trigger task that fires the interrupt in software, an ISR named `GPIOF_Handler()` that records the interrupt timing and wakes the high-priority task, a high-priority task that records its own execution timing, and a low-priority task that proves the scheduler is running continuously.

Variables Used for Verification

The watch variables used were `gInterruptCount`, `gPendSvRequests`, `gHighTaskRuns`, `gLowTaskIterations`, `gLastIsrTick`, `gLastHighTaskTick`, and `gObservedLatencyTicks`.

Comparison Strategy

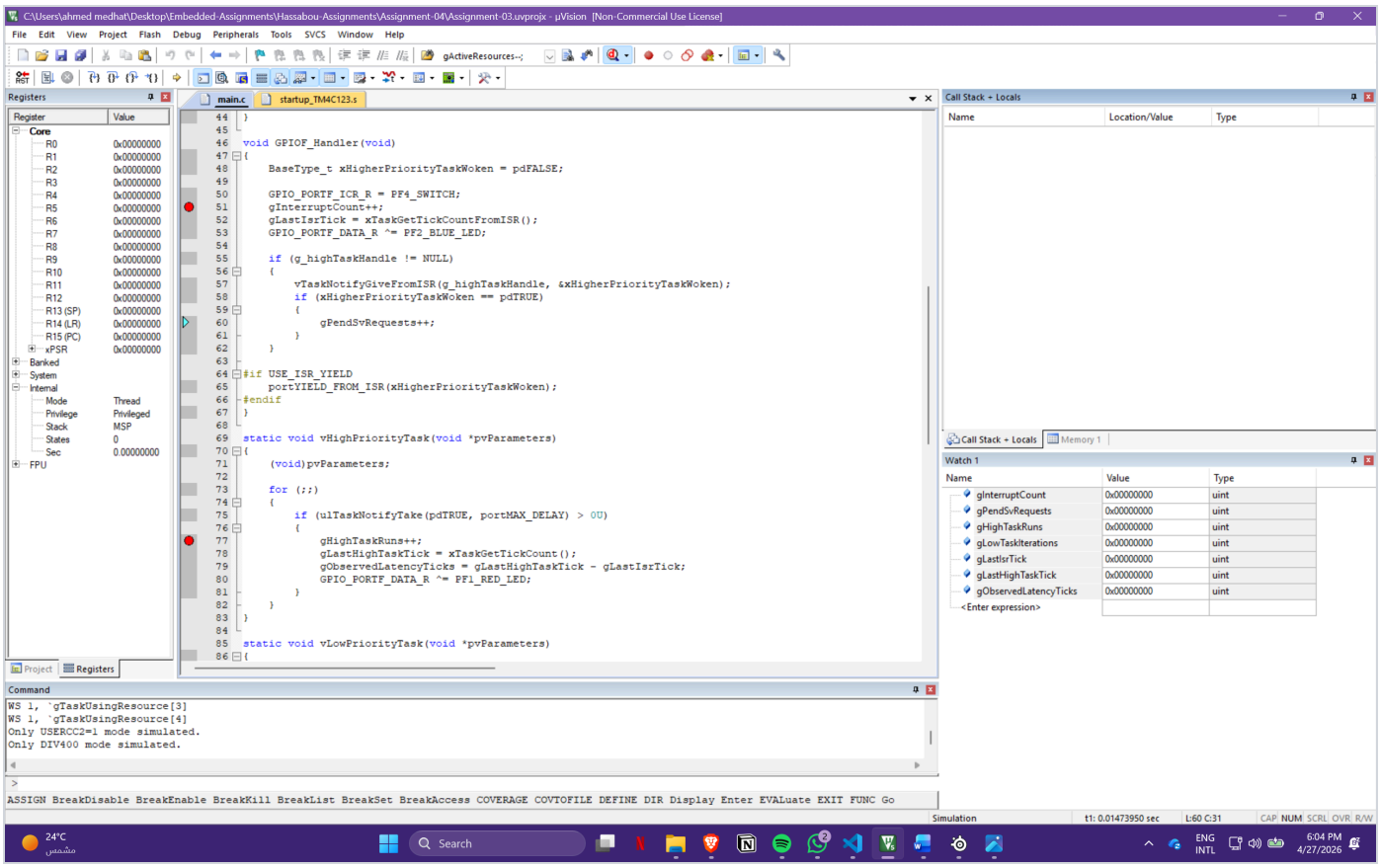
Two runs were performed. In the first run, `#define USE_ISR_YIELD 1U` was used. In the second run, `#define USE_ISR_YIELD 0U` was used. The main comparison variable was `gObservedLatencyTicks`.

Screenshot Verification and Analysis

Screenshot 1: ISR breakpoint with scheduler running

File: `Screenshot 2026-04-27 180448.png`

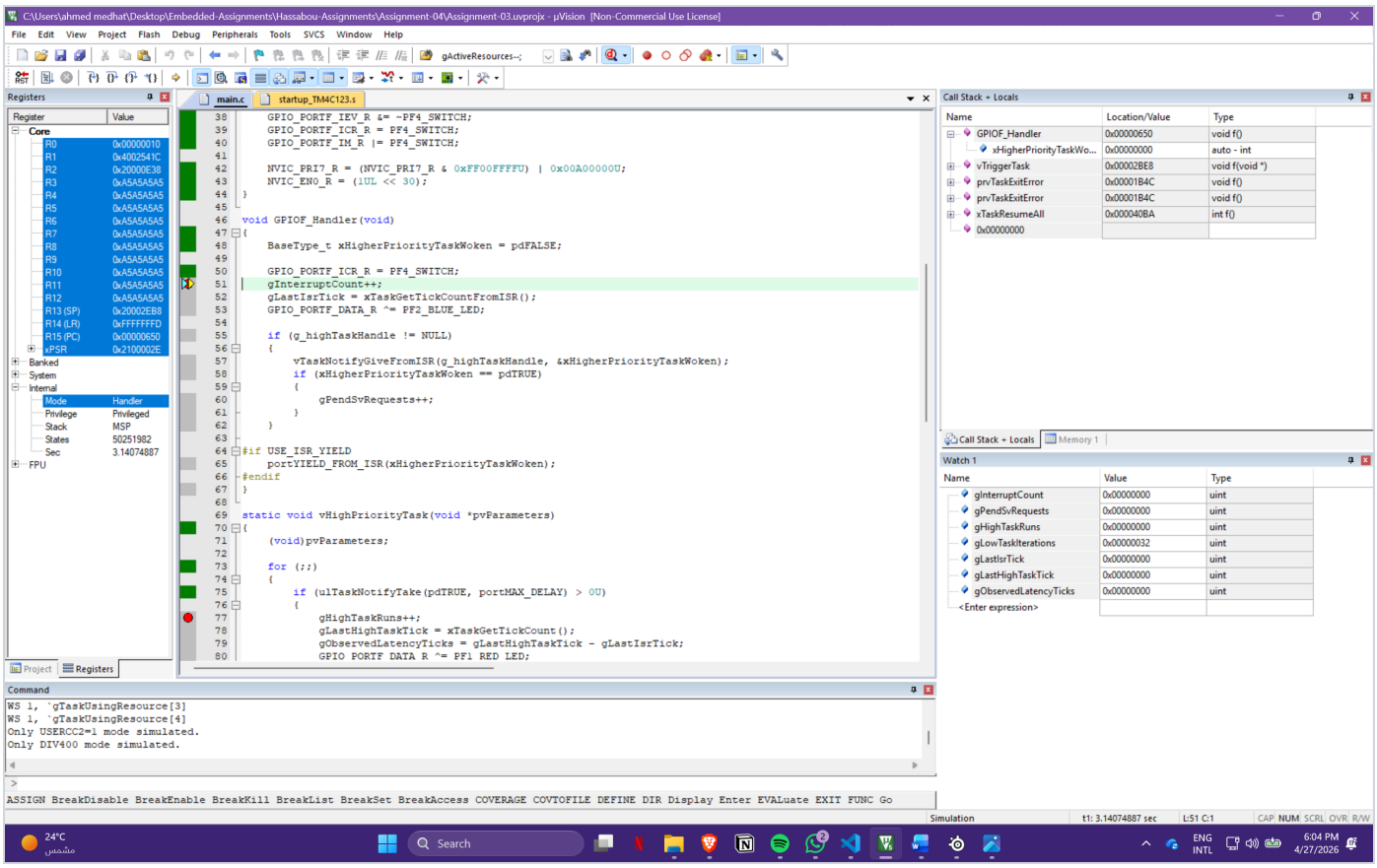
This screenshot shows the system stopped in `GPIOF_Handler()` at the ISR breakpoint. `gLowTaskIterations` is already increasing, which proves the scheduler is active and tasks are running.



Screenshot 2: ISR entry during the enabled-yield run

File: `Screenshot 2026-04-27 180457.png`

This screenshot shows the ISR entry with the original enabled-yield behavior. It is a valid setup screenshot for the `USE_ISR_YIELD = 1U` case and shows the interrupt path is being hit correctly.



Screenshot 3: Delayed task execution with `USE_ISR_YIELD = 0U`

File: `Screenshot 2026-04-27 184036.png`

This screenshot shows the delayed case after disabling `portYIELD_FROM_ISR()`. The watch values show `gInterruptCount = 1`, `gHighTaskRuns = 1`, `gLastIsrTick = 0x000003E8`, `gLastHighTaskTick = 0x000003E9`, and `gObservedLatencyTicks = 1`. This is the key proof that the high-priority task did not run immediately after the ISR.

Call Stack + Locals

Name	Location/Value	Type
GPIOF_Handler	0x0000064E	void f()
xHigherPriorityTaskWo...	0x00000000	auto - int
vTriggerTask	0x00002BC4	void f(void *)
prvTaskExitError	0x00001B28	void f()
prvTaskExitError	0x00001B28	void f()
xTaskResumeAll	0x000040B6	int f()
0x00000000		

Call Stack + Locals | Memory 1

Watch 1

Name	Value	Type
gInterruptCount	0x00000001	uint
gPendSvRequests	0x00000001	uint
gHighTaskRuns	0x00000001	uint
gLowTaskIterations	0x00000064	uint
gLastIsrTick	0x000003E8	uint
gLastHighTaskTick	0x000003E9	uint
gObservedLatencyTicks	0x00000001	uint
<Enter expression>		

Screenshot 4: Repeated delayed behavior with `USE_ISR_YIELD = 0U`

File: `Screenshot 2026-04-27 184045.png`

This screenshot confirms the same delayed behavior on the next interrupt cycle. The watch values show `gInterruptCount = 2`, `gHighTaskRuns = 2`, `gLastIsrTick = 0x000008C0`, `gLastHighTaskTick = 0x000008C1`, and `gObservedLatencyTicks = 1`. This proves the delay is consistent when the macro is disabled.

Call Stack + Locals

Name	Location/Value	Type
GPIOF_Handler	0x0000064E	void f()
xHigherPriorityTaskWo...	0x00000000	auto - int
vTriggerTask	0x00002BC4	void f(void *)
prvTaskExitError	0x00001B28	void f()
prvTaskExitError	0x00001B28	void f()
xTaskResumeAll	0x000040B6	int f()
0x00000000		

Call Stack + Locals | Memory 1

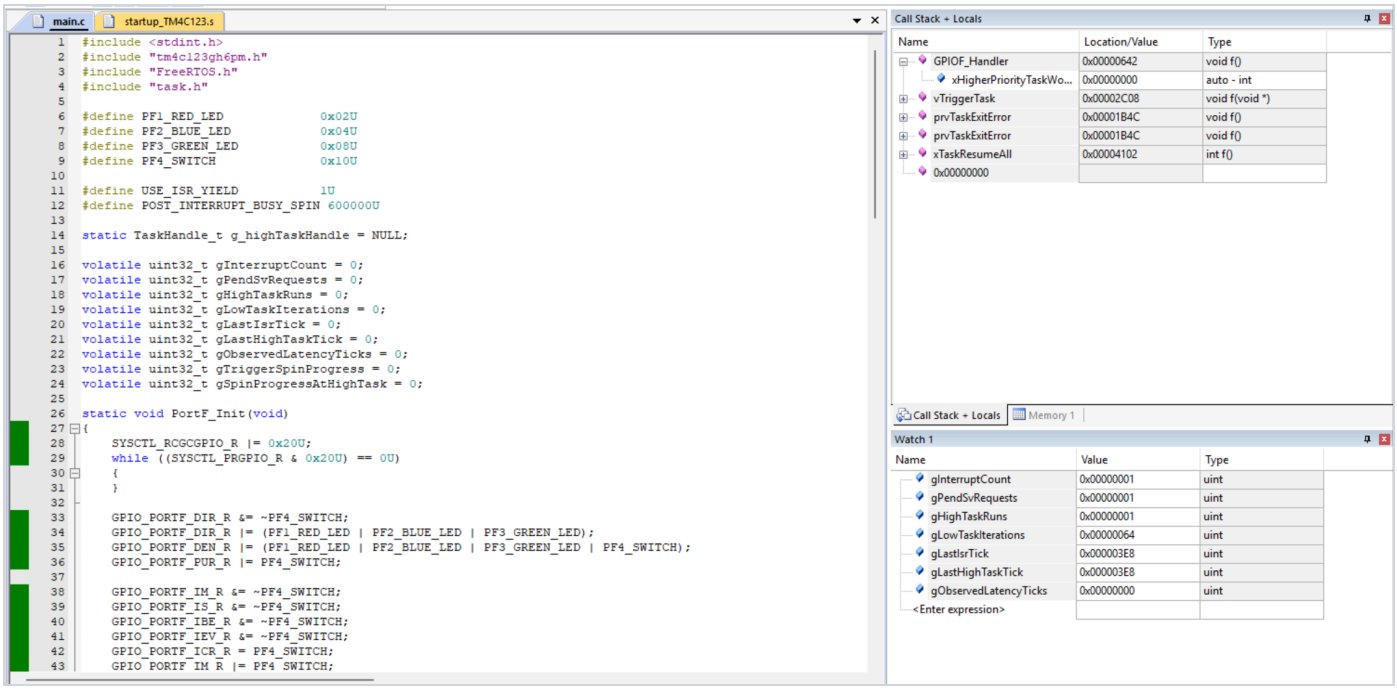
Watch 1

Name	Value	Type
gInterruptCount	0x00000002	uint
gPendSvRequests	0x00000002	uint
gHighTaskRuns	0x00000002	uint
gLowTaskIterations	0x00000096	uint
gLastIsrTick	0x000008C0	uint
gLastHighTaskTick	0x000008C1	uint
gObservedLatencyTicks	0x00000001	uint
<Enter expression>		

Screenshot 5: Immediate task execution with `USE_ISR_YIELD = 1U`

File: `Screenshot 2026-04-27 184419.png`

This screenshot shows the enabled-yield case after returning `USE_ISR_YIELD` to `1U`. The watch values show `gInterruptCount = 1`, `gHighTaskRuns = 1`, `gLastIsrTick = 0x000003E8`, `gLastHighTaskTick = 0x000003E8`, and `gObservedLatencyTicks = 0`. This is the direct contrast to the `0U` case.



Experimental Result Summary

With `portYIELD_FROM_ISR()` enabled, the high-priority task runs immediately in the same tick as the ISR and `gObservedLatencyTicks = 0`. With the macro disabled, the task runs one tick later and `gObservedLatencyTicks = 1`. This demonstrates the effect of forced context switching.

Conclusion

The assignment objective was achieved successfully. The implemented FreeRTOS application demonstrates the practical effect of `portYIELD_FROM_ISR()` on task wake-up timing. The debugger evidence proves that enabling the macro causes immediate context switching, while disabling it delays execution of the higher-priority task.